

Picking up the Red Pill

Option number two is where things are currently headed. Today, since processor speeds have reached something of a plateau, progress must be made via component integration. Sometimes a processor becomes so fast that a component that lies outside the processor becomes a hindrance, a bottleneck. Integration in this case, has a basis in solving a problem: access to memory too slow? Bring the pathways inside the processor. This was why AMD chose to integrate its processor's memory controller within the processor itself.

Integration can also be an option when there is no other path to take. As things stand, the processors have hit the thermal wall, the clockspeed party is bust and the industry is staring at a very fuzzy road ahead.

The fuzziness is reduced a bit when one follows the money.

In 2007 a powerpoint presentation made the Internet-rounds. It was shared with university students by Intel's Douglas Carmean — Chief Architect of their Visual Computing Group (VCG), in charge of GPU development at Intel. One of the slides in the presentation clearly read:

- CPU profit margins are decreasing
- GPU profit margins are increasing

And this heralded "another significant transformation."

The Road Ahead

The "significant transformation" was already in the making. It began with the introduction of the GPU. More exactly with the introduction of the NV20 chip by NVIDIA in 2001 which introduced the concept of GPU programmability. Today the floating point power of a consumer GPU card is such that it has tremendous potential to do much more than just churn out pixels.

This thinking was the basis for the so-called GPGPU, or the General Purpose GPU. NVIDIA further underlined the GP part by introducing the ability to program their graphic cards with the help of C programming language—via the CUDA (Compute Unified Device Architecture) architecture and programming model.

CUDA provides developers access to low-level GPU hardware, allowing them to use the floating-point might of the GPU to solve complex calculations to simulate smoke, fluids, and also to accelerate applications in computational biology.

Essentially, what CUDA did was bring the GPU a bit closer to the CPU. And Intel noticed this breach of territory.

If the GPU could become more CPU, then what stops the CPU to take on a GPU workload? This is Intel's point of thrust into the lucrative GPU market. By creating a processor that can handle both floating-point and scalar tasks, Intel hopes to capture the entirety of the processor market—from low-end processors used in cellphones, to high-end supercomputers churning through high-end physics simulations.

Intel is thus looking at a brave new world run on Intel platforms, or more exactly, run on Intel's "systems-on-a-chip".

A while back news broke of an initiative inside the Intel labs called the Tera-scale project. The goal of the project is to explore the possibilities of creating 'many-core' processors using current technologies as system interconnects to render future ideas such as a database-server on a chip. One possibility, for example, is that of a datacenter that is located entirely on a single microprocessor chip consisting of 80-cores—each core connected via optical fiber links and talking with each other using TCP/IP protocols.

This is an extreme example, more practical implementations though are already on the roadmap, and one such implementation is the root cause of the GPU versus CPU battle.

The Larrabee Solution

Larrabee is the code-name for a many-core architecture which is due out as engineering samples sometime by the end of 2008, and as a consumer product by 2009. A many-core processor is one that has more than eight cores, multi-core being processors with 2-8 cores. Little in the form of concrete information is known about the Larrabee, and what little was revealed by Intel, through a presentation, was later retracted when the information went public. The details are already out there, however,

The Larrabee architecture

